

Week-1

1. Install Apache Kafka on a single node.

Using Command line:

```
docker pull wurstmeister/zookeeper
```

```
docker pull wurstmeister/kafka
```

```
docker run -d --name zookeeper -p 2181:2181 wurstmeister/zookeeper
```

```
docker run -d --name kafka -p 9092:9092 --link zookeeper:zookeeper \  
-e KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 \  
-e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \  
-e KAFKA_LISTENERS=PLAINTEXT://0.0.0.0:9092 \  
wurstmeister/kafka
```

week-2

2.Demonstrate setting up a single-node, single-broker Kafka cluster and show basic operations such as creating topics and producing/consuming messages.

Run the Zookeeper and Kafka:

```
docker run -d --name zookeeper -p 2181:2181 wurstmeister/zookeeper
```

```
docker run -d --name kafka -p 9092:9092 --link zookeeper:zookeeper \  
-e KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 \  
-e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \  
-e KAFKA_LISTENERS=PLAINTEXT://0.0.0.0:9092 \  
wurstmeister/kafka
```

Generate a Cluster UUID

```
$ KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
```

Format Log Directories

```
$ bin/kafka-storage.sh format --standalone -t $KAFKA_CLUSTER_ID -c  
config/server.properties
```

Start the Kafka Server

```
$ bin/kafka-server-start.sh config/server.properties
```

Using JVM Based Apache Kafka Docker Image

Get the Docker image:

```
$ docker pull apache/kafka:4.0.0
```

Start the Kafka Docker container:

```
$ docker run -p 9092:9092 apache/kafka:4.0.0
```

Using GraalVM Based Native Apache Kafka Docker Image

Get the Docker image:

```
$ docker pull apache/kafka-native:4.0.0
```

Start the Kafka Docker container:

```
$ docker run -p 9092:9092 apache/kafka-native:4.0.0
```

STEP 3: CREATE A TOPIC TO STORE YOUR EVENTS

```
$ bin/kafka-topics.sh --create --topic quickstart-events --bootstrap-server localhost:9092
```

```
$ bin/kafka-topics.sh --describe --topic quickstart-events --bootstrap-server localhost:9092
```

```
Topic: quickstart-events      TopicId: NPmZHyhbR9y00wMglMH2sg PartitionCount: 1
ReplicationFactor: 1 Configs:
Topic: quickstart-events Partition: 0    Leader: 0    Replicas: 0 Isr: 0
```

STEP 4: WRITE SOME EVENTS INTO THE TOPIC

```
$ bin/kafka-console-producer.sh --topic quickstart-events --bootstrap-server
localhost:9092
>This is my first event
>This is my second event
```

STEP 5: READ THE EVENTS (Same Path open in Other Terminal)

Open another terminal session and run the console consumer client to read the events you just created:

```
$ bin/kafka-console-consumer.sh --topic quickstart-events --from-beginning --
bootstrap-server localhost:9092
This is my first event
This is my second event
```

You can stop the consumer client with Ctrl-C at any time.

STEP 6: IMPORT/EXPORT YOUR DATA AS STREAMS OF EVENTS WITH KAFKA CONNECT

```
$ echo "plugin.path=libs/connect-file-4.0.0.jar" >> config/connect-
standalone.properties
```

Then, start by creating some seed data to test with:

```
$ echo -e "foo\nbar" > test.txt
```

Or on Windows:

```
$ echo foo > test.txt
$ echo bar >> test.txt
```

* When you first ran consumer, **test-topic** didn't exist yet.

* After creation (or restart), it now appears in the list — so you can consume messages without error.

Next Steps (To Test End-to-End)

1. Produce Messages

Open a new terminal in the same Kafka directory:

```
bash
```

```
bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test-topic
```

```
...
```

Type:

```
...
```

Hello

Kafka Test Message

```
...
```

Press **Enter** after each line.

```
---
```

2. Consume and Save to File

Run this in another terminal:

```
bash
```

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
```

```
--topic test-topic --from-beginning > test.sink.txt
```

```
...
```

Stop after a few messages with **Ctrl + C**.

3. Verify File

``bash

cat test.sink.txt

``

Optional: Show Messages on Screen AND Save to File

``bash

bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \

--topic test-topic --from-beginning | tee test.sink.txt